

Prüfungssemester: [OFFEN]
Studiengang: User Experience Design (UXD)
Prüfungsfach: Softwareentwicklung 2 (SWE2)
Dozent: Prof. Dr. Simon Nestler

Name	Matrikelnr.	Datum
------	-------------	-------

Anmerkungen

Hinweise zur Bearbeitung

- Die Bearbeitungszeit beträgt 120 Minuten, die maximal erreichbare Punktzahl beträgt 100 Punkte. Rechnen Sie mit etwa einer Minute je Punkt und planen Sie die übrigen 20 Minuten für Lesen und Kontrolle ein.
- Diese Probeklausur entspricht in Umfang, Aufgabenschnitt und Bewertung der Klausur (schrP120).
- Bitte beantworten Sie die einzelnen Fragen direkt in dem jeweils für die Beantwortung vorgesehenen Antwortfeld.
- Der Ihnen für die Beantwortung einer Frage zur Verfügung gestellte Platz wurde anhand der erwarteten Ausführlichkeit Ihrer Beantwortung bemessen.
- **Bitte verwenden Sie keine Hilfsmittel (Entwicklungsumgebung, ChatGPT, Google, Lösung, etc.), um eine realistische Bewertung Ihres Leistungsstandes zu erhalten. Die Klausur läuft ohne Rechner.**

– bitte eigenständig (im Rahmen der Besprechung) ausfüllen –

Punkte	Prozent	Note
--------	---------	------

Durch das Unterschreiben mit meinem Namen (in Druckbuchstaben) bestätige ich, dass ich mich bei der Selbstbewertung fair und realistisch eingeschätzt habe.

Aufgabe 1 (14P): Die Werkzeugleiste einer Galerie-App verwendet die Hierarchie `Control` → `Button` → `IconButton`. Das folgende Programm kompiliert und läuft fehlerfrei (alle Klassen liegen in der Datei `Probeklausur.java`):

```
class Control { // 1
    String art = "Control"; // 2
    void render() { System.out.println("Rahmen"); } // 3
    void zeichne(Control c) { // 4
        System.out.println("zeichne Control"); // 5
    } // 6
} // 7
class Button extends Control { // 8
    String art = "Button"; // 9
    @Override void render() { System.out.println("Text"); } // 10
} // 11

class IconButton extends Button { // 12
    @Override void render() { System.out.println("Icon"); } // 13
    void zeichne(IconButton b) { // 14
        System.out.println("zeichne IconButton"); // 15
    } // 16
} // 17

public class Probeklausur { // 18
    public static void main(String[] args) { // 19
        IconButton ib = new IconButton(); // 20
        Control c = ib; // 21
        c.render(); // Zeile A // 22
        c.zeichne(ib); // Zeile B // 23
        System.out.println(c.art); // Zeile C // 24
    } // 25
} // 26
```

a) Geben Sie die vollständige Ausgabe des Programms an. (3P)

b) Zeile A: Entscheidet Java zur Übersetzungszeit oder zur Laufzeit, welche Methode läuft? Begründen Sie in einem Halbsatz, welcher Typ von `c` darüber bestimmt. (2P)

c) Zeile B: Übersetzungszeit oder Laufzeit? Begründen Sie in einem Halbsatz, warum Java genau diese `zeichne`-Fassung wählt. (2P)

d) Zeile C: Nach welcher Regel löst Java das Feld `art` auf, und welcher Typ von `c` entscheidet? Ein Halbsatz genügt. (2P)

e) Angenommen, Zeile 21 lautet `IconButton c = ib;` und der Rest bleibt unverändert. Geben Sie die neue Ausgabe an und begründen Sie jede Zeile, die sich ändert, in je einem Halbsatz. (5P)



Der Platz reicht nicht?

Bitte nutzen Sie die Rückseite des vorherigen Blattes

Aufgabe 2 (13P): Die folgende Erklärung des Programms aus Aufgabe 1 stammt aus einem KI-Werkzeug. Sie ist flüssig formuliert und enthält genau **eine** falsche Aussage:

„Das Programm nutzt Polymorphismus. (Satz 1) Die Variable `c` hat den statischen Typ `Control`, verweist aber auf ein `IconButton`-Objekt. (Satz 2) Beim Aufruf `c.render()` entscheidet deshalb der dynamische Typ: Es läuft die Fassung aus `IconButton`, die Ausgabe ist ‚Icon‘. (Satz 3) Beim Aufruf `c.zeichne(ib)` wählt der Compiler die Methode anhand des statischen Typs, sodass `zeichne(Control)` läuft und ‚zeichne Control‘ erscheint. (Satz 4) Auch der Zugriff `c.art` folgt dem Objekt: Da im Speicher ein `IconButton` liegt, wird das Feld der tatsächlichen Klasse verwendet, die Ausgabe ist ‚Button‘. (Satz 5)“

a) Benennen Sie den falschen Satz (Satznummer genügt). Reine Stilkritik zählt nicht. (3P)

b) Nennen Sie die Bindungsregel, die den Satz widerlegt, und formulieren Sie sie in einem vollständigen Satz. (4P)

c) Nennen Sie die Programmzeile aus Aufgabe 1, mit der Sie die Widerlegung beweisen, und geben Sie an, was diese Zeile tatsächlich ausgibt. (4P)

d) Woran erkennen Sie auch ohne dieses Beispiel, ob eine Code-Erklärung belastbar oder bloß plausibel ist? Nennen Sie zwei allgemeine Merkmale. (2P)



Der Platz reicht nicht?

Bitte nutzen Sie die Rückseite des vorherigen Blattes

Aufgabe 3 (14P): Die Galerie-App lädt beim Start ein Bild. Die Methode `BildLader.laden(String pfad)` liest die Datei und deklariert `throws IOException`; fehlt die Datei, wirft sie eine `FileNotFoundException`. Auf dem Rechner existiert die Datei `urlaub/strand.jpg` **nicht**. Das folgende Programm kompiliert:

```
public class Galerie {                                // 1
    static byte[] daten;                              // 2
    public static void main(String[] args) {          // 3
        try {                                         // 4
            daten = BildLader.laden("urlaub/strand.jpg"); // 5
        } catch (Exception e) {                      // 6
        }                                             // 7
        System.out.println(daten.length + " Bytes geladen"); // 8
    }                                                 // 9
}                                                    // 10
```

Beim Ausführen erscheint dieses Fehlerbild:

```
Exception in thread "main" java.lang.NullPointerException: Cannot read the array length
because "Galerie.daten" is null
    at Galerie.main(Galerie.java:8)
```

- a) Die Datei fehlt. Warum meldet das Programm trotzdem eine `NullPointerException` in Zeile 8 und keine `FileNotFoundException` aus Zeile 5? Erklären Sie den Ablauf. (4P)

- b) Das Fehlerbild zeigt auf Zeile 8, die Ursache liegt woanders. Nennen Sie die gefährlichste Stelle des Programms (Zeilennummer) und begründen Sie, warum ausgerechnet sie das Fehlerbild auf die falsche Zeile lenkt. (2P)

- c) Schreiben Sie den `try/catch`-Teil so um, dass das Programm eine fehlende Datei behandelt und nicht mehr abstürzt. Die Erfolgsmeldung erscheint nur bei Erfolg; im Fehlerfall gibt das Programm eine Konsolenmeldung mit dem Grund aus. (5P)

- d) Machen Sie aus der Konsolenmeldung eine Fehlermeldung für die Benutzerin. Ihr Text beantwortet drei Fragen: Was ist passiert? Was ist die Folge? Was kann ich jetzt tun? (3P)



Der Platz reicht nicht?

Bitte nutzen Sie die Rückseite des vorherigen Blattes

Aufgabe 4 (14P): Ein Fotoarchiv verwaltet Bildnummern in einem binären Suchbaum:

```
class Knoten {
    int wert;
    Knoten links, rechts;
    Knoten(int wert) { this.wert = wert; }
}

static Knoten einfuegen(Knoten k, int wert) {
    if (k == null) return new Knoten(wert);
    if (wert < k.wert) k.links = einfuegen(k.links, wert);
    else k.rechts = einfuegen(k.rechts, wert);
    return k;
}

static boolean enthaelt(Knoten k, int wert) {
    if (k == null) return false;
    if (wert == k.wert) return true;
    if (wert < k.wert) return enthaelt(k.links, wert);
    return enthaelt(k.rechts, wert);
}
```

- a) Die Werte 7, 3, 9, 1, 4 werden in dieser Reihenfolge in einen leeren Baum eingefügt. Zeichnen Sie den entstehenden Baum. (3P)

- b) Welche Knoten besucht der Aufruf `enthaelt(wurzel, 4)` im Baum aus Teilaufgabe a)? Geben Sie sie in der Reihenfolge an, in der sie besucht werden. (3P)

- c) Geben Sie eine Einfügereihenfolge derselben fünf Werte an, bei der der Baum zur Kette entartet. Was bedeutet das für die Suche? Ein Satz genügt. (4P)

- d) Implementieren Sie die Methode `static int zaehle(Knoten k)`, die rekursiv die Anzahl der Knoten im Baum zurückgibt (leerer Baum: 0). (4P)



Der Platz reicht nicht?

Bitte nutzen Sie die Rückseite des vorherigen Blattes

Aufgabe 5 (15P): Die Galerie-App verwaltet Medien. Zwei Entwürfe stehen zur Wahl. **Entwurf (a):** Oberklasse `Medium` mit den Methoden `getTitel()` und `abspielen()`, Unterklassen `Foto` und `Video`. **Entwurf (b):** Klasse `Medium` mit `getTitel()`, dazu ein Interface `Abspielbar` mit der Methode `abspielen()`; `Video` erbt von `Medium` und implementiert `Abspielbar`, `Foto` erbt nur von `Medium`. Im nächsten Release kommen `Audio` (abspielbar) und `Livestream` (abspielbar) hinzu.

- a) Entwurf (a) hat mit der Klasse `Foto` ein Problem. Benennen Sie es in einem Satz. (3P)

- b) Skizzieren Sie Entwurf (b) in Java: Deklarieren Sie das Interface `Abspielbar` und schreiben Sie die Klassenköpfe von `Video` und `Audio`, jeweils mit der Methode `abspielen()`. Die Methodenrumpfe dürfen leer bleiben oder nur einen Kommentar enthalten. (4P)

- c) Entscheiden Sie sich für einen der beiden Entwürfe und nennen Sie das Kriterium, das für Sie den Ausschlag gibt. (4P)

- d) Nennen Sie den Preis Ihres gewählten Entwurfs: Welchen konkreten Nachteil nehmen Sie in Kauf? (4P)



Der Platz reicht nicht?

Bitte nutzen Sie die Rückseite des vorherigen Blattes

Aufgabe 6 (15P): Eine Vokabel-Lern-App speichert Stapel, Karten und Lernstände mehrerer Personen: Eine Karte hat Vorder- und Rückseite, ein Stapel bündelt Karten, und die App hält für jede Person je Karte fest, wie sicher sie sie beherrscht (Lernstufe 0–5).

- a) Skizzieren Sie das Klassenmodell mit den Klassen, ihren wichtigsten Attributen und den Beziehungen zwischen ihnen. Skizze oder Stichpunkte genügen, Java-Code ist nicht nötig. (6P)

- b) Begründen Sie in zwei bis drei Sätzen: Warum ist der Lernstand eine eigene Klasse und kein Attribut der Karte? (3P)

- c) Implementieren Sie die Klasse `Karte` mit privaten Attributen `vorderseite` und `rueckseite`, einem Konstruktor, der beide setzt, und Gettern für beide Attribute. (3P)

- d) Angenommen, `vorderseite` wäre `public`: Geben Sie eine konkrete Codezeile an, die den Zustand einer Karte von außen kaputtmacht, und nennen Sie in einem Halbsatz, was die Kapselung hier verhindert. (3P)



Der Platz reicht nicht?

Bitte nutzen Sie die Rückseite des vorherigen Blattes

Aufgabe 7 (15P): Die Galerie-App bekommt die Funktion „Album schärfen“: Ein Klick auf den Button `schaerfen` wendet einen Schärfefilter auf alle 600 Fotos eines Albums an und speichert jedes Ergebnis. `balken` ist ein Fortschrittsbalken (`ProgressBar`, Werte von 0 bis 1), `status` ein `Label` als Statuszeile der Oberfläche. Der Handler kompiliert und arbeitet fehlerfrei, am Ende sind alle Fotos geschärft. Trotzdem friert die Oberfläche nach dem Klick ein, bis das letzte Foto bearbeitet ist: Der Balken bewegt sich nicht, weitere Klicks bleiben ohne Reaktion.

```
schaerfen.setOnAction(e -> {                                // 1
    schaerfen.setDisable(true);                               // 2
    List<Foto> fotos = album.fotos();                         // 3
    for (int i = 0; i < fotos.size(); i++) {                  // 4
        Foto neu = filter.anwenden(fotos.get(i));            // 5 ~40 ms
        archiv.speichern(neu);                               // 6 ~10 ms
        balken.setProgress((i + 1.0) / fotos.size());        // 7
    }                                                         // 8
    status.setText("Album geschärft");                        // 9
    schaerfen.setDisable(false);                             // 10
});                                                         // 11
```

- a) Warum friert die Oberfläche ein, und warum ändert Zeile 7 daran nichts, obwohl sie den Balken in jedem Durchlauf setzt? Erklären Sie beides in höchstens drei Sätzen. (4P)

- b) Skizzieren Sie den Umbau in Worten oder Code: Welche Zeilen laufen danach im Hintergrund, welche bleiben im Event-Thread (JavaFX Application Thread), und auf welchem Weg gelangen Fortschritt und Fertig-Meldung zurück in die Oberfläche? Die Namen aus der Vorlesung (`Task`, `updateProgress`, `setOnSucceeded`, `Platform.runLater`) dürfen Sie verwenden, verlangt sind sie nicht. (6P)

- c) Entwerfen Sie die Rückmeldung für die Wartezeit: Nennen Sie die Stufe der Wartekommunikation, begründen Sie Ihre Wahl an der Dauer des Vorgangs und nennen Sie den Preis Ihrer Wahl. (5P)



Der Platz reicht nicht?

Bitte nutzen Sie die Rückseite des vorherigen Blattes